

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I P.Harika(192311243) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

P.Harika

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Servlet Architecture

A Servlet is a Java program that runs on a web server and processes requests from a web browser.

Architecture:

Student → Browser → Web Server → Servlet → Response → Browser

- **Browser:** Student enters registration details.
- **Web Server:** Receives the request.
- **Servlet:** Processes the submitted data.

- **Response:** Result is sent back to the browser.

2. HTML Registration Form

```
<form action="StudentServlet" method="post">
  Name: <input type="text" name="name"><br>
  Email: <input type="email" name="email"><br>
  Course: <input type="text" name="course"><br>
  Password: <input type="password" name="password"><br>
  <input type="submit" value="Register">
</form>
```

3. Java Servlet

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class StudentServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req,
                          HttpServletResponse res)
        throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        String name = req.getParameter("name");
        out.println("<h2>Student Name: " + name + "</h2>");
    }

    protected void doPost(HttpServletRequest req,
                          HttpServletResponse res)
        throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        String name = req.getParameter("name");
        String email = req.getParameter("email");
```

```

String course = req.getParameter("course");

out.println("<h2>Registration Successful</h2>");
out.println("Name: " + name + "<br>");
out.println("Email: " + email + "<br>");
out.println("Course: " + course);
}
}

```

4. Servlet Life Cycle

The servlet life cycle has three main methods:

- **init()** – Called once when the servlet is created.
- **service()** – Handles each client request and calls doGet() or doPost().
- **destroy()** – Called when the servlet is removed from the server.

init() → service() → doGet()/doPost() → destroy()

5. GET vs POST

GET

Data appears in URL

Used mainly for retrieving data

Not suitable for sensitive data

POST

Data is sent in request body

Used for submitting data

More suitable for sensitive data

Conclusion: **POST** is preferred for sensitive information such as passwords because the data is not displayed in the URL. For complete security, **HTTPS** should also be used.

2. 1. Introduction

Session management is used to maintain a user's login information across multiple pages. After successful login, the server creates a session for the user.

2. Login Form

```

<form action="LoginServlet" method="post">
    Username: <input type="text" name="username"><br>
    Password: <input type="password" name="password"><br>
    <input type="submit" value="Login">
</form>

```

3. Login Servlet Using HttpSession

```

import java.io.*;
import jakarta.servlet.*;

```

```

import jakarta.servlet.http.*;

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req,
                           HttpServletResponse res)
        throws IOException {

        String username = req.getParameter("username");
        String password = req.getParameter("password");

        if (username.equals("admin") && password.equals("1234")) {

            HttpSession session = req.getSession();
            session.setAttribute("username", username);

            res.sendRedirect("home.jsp");

        } else {
            res.getWriter().println("Invalid Login");
        }
    }
}

```

4. Session vs Cookies

Session

Data is mainly stored on the server

More suitable for login sessions

Uses a session ID

Can store more information

More secure for sensitive session data

Cookies

Data is stored in the browser

Suitable for small information

Stores key-value data

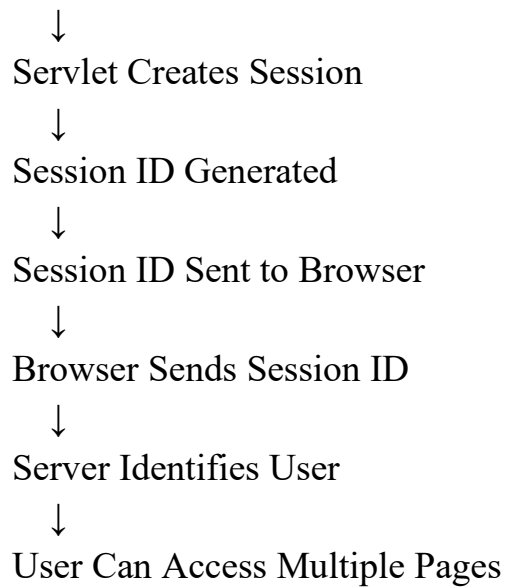
Limited storage

Conclusion

HttpSession is preferred for maintaining login sessions because user information is stored on the server. **Cookies** are useful for storing small client-side information and are commonly used to carry the session ID.

5. How Session Tracking Works

User Login



Conclusion

HttpSession is preferred for maintaining login sessions because user information is stored on the server. **Cookies** are useful for storing small client-side information and are commonly used to carry the session ID.

3. 1. Introduction

JDBC (Java Database Connectivity) is used to connect a Java application with a database. In an e-commerce application, JDBC can be used to **insert and retrieve product details**.

2. JDBC Architecture

Java Application



JDBC API



JDBC Driver



Database

- ❑ **Java Application:** Sends database requests.
- ❑ **JDBC API:** Provides classes and methods for database operations.
- ❑ **JDBC Driver:** Connects Java with the specific database.
- ❑ **Database:** Stores product information.

3. Product Table

```
CREATE TABLE product (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    price DOUBLE  
);
```

4. JDBC Program

```
import java.sql.*;
```

```
public class ProductDB {
```

```
    public static void main(String[] args) {
```

```
        String url = "jdbc:mysql://localhost:3306/shop";
```

```
        String user = "root";
```

```
        String password = "1234";
```

```
        try {
```

```
            // 1. Connect to database
```

```

Connection con = DriverManager.getConnection(
    url, user, password);

// 2. Insert product
String insert = "INSERT INTO product VALUES (?, ?, ?)";
PreparedStatement ps = con.prepareStatement(insert);

ps.setInt(1, 101);
ps.setString(2, "Laptop");
ps.setDouble(3, 50000);

ps.executeUpdate();

// 3. Retrieve products
Statement st = con.createStatement();
ResultSet rs = st.executeQuery(
    "SELECT * FROM product");

while (rs.next()) {
    System.out.println(
        rs.getInt("id") + " " +
        rs.getString("name") + " " +
        rs.getDouble("price"));
}

// 4. Close connection
con.close();

} catch (SQLException e) {
    System.out.println("Database Error: " + e.getMessage());
}
}
}

```

5. Steps in JDBC Connectivity

- 1. Load JDBC Driver**
- 2. Establish database connection**
- 3. Create Statement/PreparedStatement**

4. **Execute SQL query**
5. **Process the ResultSet**
6. **Close the connection**

6. Exception Handling

JDBC operations may cause errors such as incorrect database details or SQL errors. These are handled using try-catch.

```
catch (SQLException e) {  
    System.out.println("Database Error: " + e.getMessage());  
}
```

Conclusion

JDBC provides a simple way for a Java web application to communicate with a database. It can be used to **insert, retrieve, update, and delete product information** using SQL queries. PreparedStatement is preferred for secure database operations.